From Data to Machine Learning with Python

Modul pembelajaran lengkap dengan ilustrasi dan pembahasan praktikum

Modul pembelajaran peserta

Data dapat membantu kita menjawab pertanyaan yang sulit diselesaikan dengan pengamatan satu per satu. Sebuah toko ingin mengenali pola pembelian, petugas layanan ingin mengelompokkan pesan, dan peneliti ingin memahami hubungan antara atribut yang tercatat dan label suatu kasus. Machine learning menyediakan cara untuk mempelajari pola dari contoh, kemudian menggunakannya pada masukan baru.

Modul ini membahas perjalanan dari masalah hingga aplikasi sederhana. Anda akan mengenali jenis tugas machine learning, menggunakan CRISP-DM untuk merumuskan pekerjaan, mengubah data menjadi fitur numerik, melatih classifier, membaca kesalahan prediksi, dan menjalankan model melalui Streamlit. Contoh menggunakan data tabular, teks dan citra; pemodelan inti difokuskan pada klasifikasi tabular.

Apa yang akan Anda hasilkan

Hasil pembelajaran berupa pipeline k-NN untuk data pembelian, pipeline decision tree untuk studi kasus preeklamsia, tabel evaluasi, file model tersimpan, dan aplikasi yang menerima masukan melalui formulir. Notebook memuat kode, penjelasan, visualisasi dan hasil yang dapat ditelusuri dari data hingga aplikasi.

Pengetahuan awal

Anda cukup mengenal variabel, list, operasi hitung dan cara menjalankan Python. Penjelasan tentang array, DataFrame, fitur dan target disertakan sebelum pemodelan. Kode dapat dijalankan bertahap; pahami bentuk data dan hasil setiap langkah sebelum mengubah parameter.

Hubungan materi dan berkas

Materi	Berkas pembelajaran
Python dan representasi	Notebook 01 dan 02 pada folder praktikum
Preprocessing dan klasifikasi	Notebook 03 dan 04 pada folder praktikum
Kasus preeklamsia	Notebook 05 sampai 07 pada folder studi_kasus
Inferensi dan aplikasi	Notebook 08 pada deployment/Notebook_08
Pengayaan teks	Notebook 09 pada folder praktikum

Pelajari konsep, ikuti contoh pada notebook 01–08, kemudian kerjakan latihan untuk memeriksa pemahaman. Notebook 09 menyediakan pengayaan klasifikasi teks. Semua dataset yang digunakan tersedia di dalam paket. Data lengkap tetap dibaca meskipun tampilan contoh hanya memperlihatkan beberapa baris.

Peta materi dan praktikum

Alur belajar bergerak dari alasan menggunakan ML menuju cara menyajikan model dalam aplikasi. Pada setiap tahap, perhatikan pertanyaan yang dijawab oleh kode. Kode yang berhasil berjalan belum cukup; Anda perlu mengetahui data apa yang masuk, apa yang dipelajari, dan bagaimana keluaran dinilai.

Notebook	Data dan kegiatan	Hasil yang dipahami
01 Python dan eksplorasi	Tab.csv dan Social_Network_Ads.csv	Array, tabel, statistik dan grafik
02 Input dan feature space	Data.csv, teks sentimen dan fruits.png	Representasi numerik tiga jenis data
03 Preprocessing	Data.csv lengkap 10 baris	Split 7 train dan 3 test; imputasi dan encoding
04 Klasifikasi kNN	Social_Network_Ads.csv 400 baris	CV untuk memilih k; baseline dan test
05 Memahami data PE	dataKasus-1.xlsx 561 baris	Audit atribut, target dan kualitas data
06 Pipeline dan evaluasi PE	8 prediktor dan target PE	CV train, pohon terpilih dan metrik test
07 Ekspor model	Konfigurasi terpilih pada 06	Reproduksi, checksum dan model card
08 Inferensi dan Streamlit	Model tersimpan dan contoh input	Hubungan masukan aplikasi dan prediksi
09 Pengayaan teks	Dataset_Sentimen_Emosi.csv	TF-IDF dan klasifikasi Naive Bayes

Cara mengikuti satu contoh sampai selesai

Baca konsep terlebih dahulu, amati gambar atau tabel, lalu jalankan sel notebook yang sesuai. Setelah melihat output, jelaskan dengan kalimat sendiri apa yang berubah. Misalnya, encoding mengubah bentuk kolom, sedangkan split memisahkan baris. Perbedaan ini membantu Anda menemukan kesalahan saat bentuk matriks tidak sesuai.

Cuplikan kode dalam modul menyoroti bagian yang sedang dipelajari. Kode bertanda ilustrasi menggunakan data kecil agar bisa dihitung manual. Notebook dalam paket memuat import, lokasi berkas dan penyimpanan hasil secara lengkap. Jalankan sel dari atas ke bawah agar variabel dan model terbentuk dalam urutan yang benar.

Berkas dan lingkungan belajar

Notebook 01–04 dan 09 berada di praktikum/notebooks. Notebook 05–07 berada di studi_kasus/notebooks. Notebook 08 berada di deployment/Notebook_08. Nama berkas notebook menggunakan akhiran _Modul.ipynb, misalnya 08_Inferensi_dan_Streamlit_Modul.ipynb. Angka pada nama notebook menyatakan urutan kegiatan, bukan nomor bab modul.

Gunakan Python 3.10 dan dependensi dari requirements.txt pada folder utama. Pilih kernel Jupyter pada environment yang sama. Model bawaan versi 2 memakai scikit-learn 1.7.1. Notebook 03 menyimpan hasil ke hasil_praktikum_03 di direktori kerja saat dijalankan; notebook 08 menyimpan hasil ke hasil_praktikum08 pada folder utama.

Memahami machine learning

Belajar pola dari contoh

Pada program berbasis aturan, pengembang menulis kondisi secara langsung, misalnya jika stok kurang dari batas tertentu maka tampilkan peringatan. Pada machine learning, pengembang menyiapkan contoh dan tujuan belajar, kemudian algoritma membentuk pola atau fungsi keputusan dari data. Model yang dihasilkan tetap perlu diuji karena pola pada data lama belum tentu berlaku pada data baru.

Bayangkan tersedia catatan usia, estimasi penghasilan dan keputusan pembelian. Setiap catatan menjadi satu sampel. Usia serta penghasilan menjadi fitur, sedangkan membeli atau tidak membeli menjadi label. Algoritma klasifikasi mempelajari hubungan fitur dan label; model kemudian digunakan untuk memperkirakan kelas sampel baru.

Sampel fitur target dan model

Istilah	Makna	Contoh
Sampel	Satu objek atau kejadian yang diamati	Satu baris catatan pelanggan
Fitur X	Informasi masukan bagi model	Usia dan estimasi penghasilan
Target y	Jawaban yang ingin dipelajari	Purchased bernilai 0 atau 1
Algoritma	Prosedur untuk mempelajari pola	k-NN atau decision tree
Model	Objek hasil proses pelatihan	Pipeline yang sudah menjalankan fit
Prediksi	Keluaran model untuk masukan	Kelas membeli pada sampel baru

Training dan inferensi

Training adalah proses membangun model menggunakan data latih. Pada supervised learning, data latih menyediakan fitur dan label. Inferensi adalah proses menggunakan model yang telah terbentuk untuk menghasilkan prediksi. Label benar sampel baru biasanya belum diketahui ketika inferensi dilakukan.

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

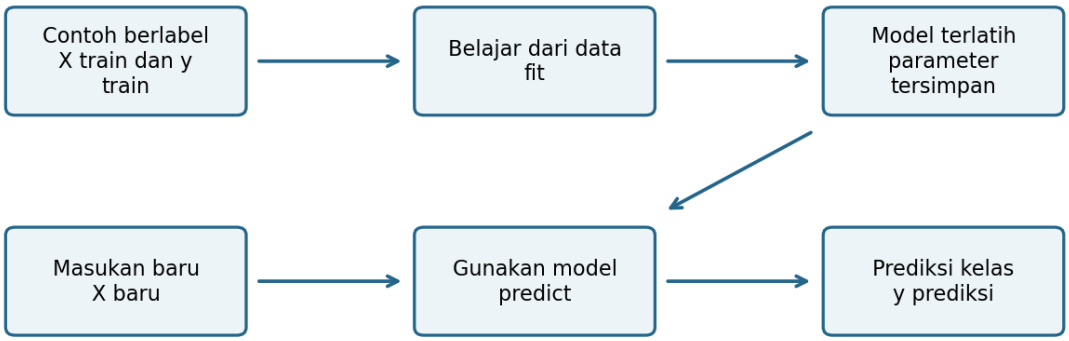
fit mempelajari pola, sedangkan predict menggunakan pola tersebut. Nama X_test menunjukkan bahwa fitur ini disediakan untuk evaluasi. Label y_test disimpan sebagai pembanding; label tersebut tidak diberikan kepada model ketika predict dipanggil.

Jenis tugas

Klasifikasi menghasilkan kategori, misalnya membeli atau tidak membeli. Regresi menghasilkan besaran numerik, misalnya nilai penjualan. Clustering mencari kelompok dari pola kemiripan tanpa label target yang ditentukan sebelumnya. Workshop ini menggunakan supervised learning untuk klasifikasi.

Latihan: tentukan jenis tugas untuk mengelompokkan keluhan berdasarkan kemiripan, memprediksi jumlah pengunjung, dan mengenali jenis produk dari gambar. Jawabannya berturut-turut adalah clustering, regresi dan klasifikasi, dengan asumsi label jenis produk tersedia untuk pelatihan.

Melihat proses belajar dan memprediksi



Gambar 1 Dua proses yang menggunakan data dengan peran berbeda. Label target diberikan saat training dan digunakan sebagai pembanding saat evaluasi.

Langkah 1 Menyediakan pengalaman belajar

Pada pembelajaran terawasi, satu contoh berisi masukan dan jawaban yang diketahui. Dalam data pembelian, usia dan penghasilan adalah masukan, sementara Purchased adalah jawaban. Jika contoh hanya berisi usia dan penghasilan tanpa keputusan pembelian, kita belum memiliki pasangan X dan y untuk melatih classifier pembelian.

Langkah 2 Membentuk model

Perintah fit memberi algoritma kesempatan mempelajari hubungan pada data latih. Cara belajarnya berbeda antar algoritma. k-NN menyimpan contoh dan menggunakan kedekatan untuk memprediksi, sedangkan decision tree membentuk rangkaian pemisahan berdasarkan fitur. Keduanya menghasilkan objek yang dapat dipakai kembali.

Langkah 3 Menggunakan model pada sampel baru

Saat predict dipanggil, model menerima fitur tanpa diberi jawaban benarnya. Dalam evaluasi, jawaban sebenarnya sudah kita simpan sebagai y_test. Sesudah prediksi selesai, barulah y_pred dibandingkan dengan y_test. Inilah alasan target tidak boleh ikut dimasukkan ke X.

Mengapa hasil pada data latih belum cukup

Mahasiswa yang menghafal jawaban latihan belum tentu mampu menjawab soal baru. Demikian pula model dapat cocok dengan data latih tetapi gagal pada sampel yang belum dilihat. Tujuan evaluasi adalah mengukur kemampuan tersebut dengan pemisahan data yang masuk akal.

Dalam kerangka pengalaman, tugas dan ukuran kinerja, pengalaman kita adalah contoh berlabel; tugasnya klasifikasi; ukuran kinerjanya ditentukan dari kesalahan prediksi. Data lebih banyak dapat membantu, tetapi tidak menjamin perbaikan jika label salah atau contoh tidak mewakili penggunaan.

Hubungkan ke notebook 04. Cari baris fit dan predict. Sebutkan variabel yang membawa jawaban benar pada kedua tahap tersebut.

Dari persoalan nyata menjadi tugas ML

Pemilihan algoritma dimulai setelah persoalan dipahami. Pernyataan “ingin memakai AI” belum menjelaskan apa yang perlu diprediksi, data apa yang tersedia, atau tindakan apa yang terbantu. Rumusan yang lebih jelas menghubungkan pengguna, masukan, keluaran dan penggunaan hasil.

Konteks	Masukan	Target atau keluaran	Penggunaan hasil
Layanan kampus	Teks pertanyaan	Kategori layanan	Membantu pengalihan pesan
Usaha digital	Atribut interaksi	Respons pembelian	Membantu prioritas penawaran
Industri kreatif	Gambar aset	Jenis atau kategori aset	Membantu pencarian koleksi
Penelitian kesehatan	Atribut yang tercatat	Label kasus pada dataset	Menguji pola dan keterbatasan model

Rumusan yang dapat dikerjakan

Contoh: “Kami ingin membantu petugas layanan mengarahkan pertanyaan mahasiswa dengan mengklasifikasikan satu pesan menjadi kategori akademik, keuangan atau teknis berdasarkan teks pesan.” Rumusan ini sudah menetapkan pengguna, unit sampel, fitur dan target. Langkah berikutnya adalah menilai ketersediaan contoh berlabel dan akibat kesalahan pengalihan.

Untuk data pembelian, tugas latihan adalah mengklasifikasikan Purchased berdasarkan Age dan EstimatedSalary. Model tidak otomatis menjelaskan mengapa seseorang membeli. Prediksi, penjelasan sebab akibat dan keputusan bisnis merupakan hal yang berbeda; tujuan latihan ini adalah prediksi kelas pada data yang tersedia.

Fitur harus tersedia saat dibutuhkan

Jika tujuan model adalah memperkirakan pembelian sebelum transaksi, nomor struk yang baru muncul sesudah pembelian tidak layak menjadi fitur. Informasi tersebut dapat mengungkap jawaban secara tidak langsung. Kesalahan seperti ini disebut kebocoran informasi atau data leakage.

Prinsip yang sama berlaku pada data kesehatan. Bila model dimaksudkan untuk prediksi sebelum suatu kejadian, fitur harus dicatat sebelum kejadian itu. Atribut yang diisi sesudah diagnosis tidak boleh diperlakukan sebagai bukti kemampuan prediksi dini.

Nilai keterampilan ML

Keterampilan yang dibangun mencakup pemahaman kebutuhan, pengelolaan data, pemrograman, evaluasi dan integrasi aplikasi. Kemampuan menjelaskan mengapa suatu fitur digunakan dan bagaimana kesalahan model dinilai sama pentingnya dengan menjalankan algoritma. Proyek kecil yang dapat direproduksi membantu membangun dasar untuk analisis data, aplikasi AI dan penelitian.

Latihan: tuliskan satu masalah di lingkungan Anda dengan pola “membantu siapa, memprediksi apa, dari informasi apa, untuk tindakan apa”. Pastikan setiap informasi masukan memang tersedia pada saat prediksi diperlukan.

CRISP DM untuk menyusun proses

CRISP-DM adalah kerangka kerja yang membagi proses analisis data menjadi enam tahap. Kerangka ini membantu menghubungkan kebutuhan dengan pekerjaan teknis. Prosesnya dapat berulang: hasil evaluasi mungkin menunjukkan bahwa label perlu diperbaiki, fitur perlu ditelaah, atau tujuan awal perlu dirumuskan kembali.

Tahap	Makna	Contoh pada klasifikasi pembelian
Business understanding	Memahami tujuan dan ukuran keberhasilan	Mengklasifikasikan respons pembelian untuk membantu analisis
Data understanding	Mengenali sumber, atribut dan kualitas data	Periksa label, nilai kosong dan distribusi usia
Data preparation	Membentuk data yang dapat digunakan model	Pisahkan subset, imputasi dan scaling
Modeling	Melatih algoritma dengan data latih	Latih k-NN dan baseline mayoritas
Evaluation	Menilai hasil terhadap tujuan	Baca confusion matrix dan kesalahan tiap kelas
Deployment	Menyediakan hasil untuk penggunaan	Formulir yang memanggil model tersimpan

Hubungan antar tahap

Pemahaman data dapat mengubah rumusan masalah. Misalnya, informasi yang diperlukan untuk memprediksi sebelum transaksi ternyata hanya tersedia sesudah transaksi. Dalam keadaan itu, tujuan atau sumber data harus diperbaiki sebelum training. Menambah kompleksitas algoritma tidak menyelesaikan ketidaksesuaian waktu informasi.

Evaluasi juga dapat membawa proses kembali ke persiapan data. Jika banyak kategori baru muncul saat pengujian, kemungkinan pembagian data atau representasi kategori belum mencerminkan kondisi penggunaan. Deployment kemudian memberi umpan balik tentang kelengkapan input dan perubahan distribusi data.

Contoh pada studi kasus preeklamsia

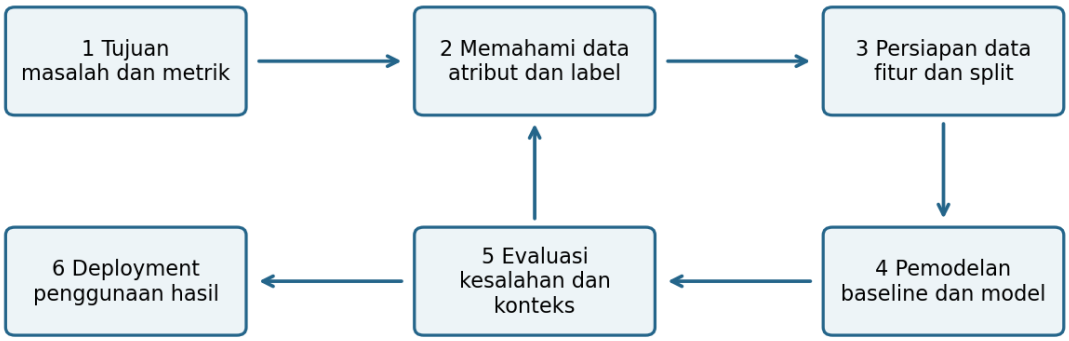
Business understanding pada latihan adalah mengklasifikasikan label PE dalam data yang tersedia. Data understanding menemukan kelas yang tidak seimbang, format usia yang beragam dan nilai kosong. Data preparation membersihkan format, memisahkan train dan test, lalu mempelajari imputasi serta encoding dari train. Modeling menggunakan pohon keputusan; evaluation membandingkannya dengan tebakan mayoritas; deployment menghubungkan pipeline ke Streamlit.

Contoh tersebut menunjukkan bahwa alur teknis selalu memiliki alasan. Imputasi diperlukan karena ada data kosong, one hot diperlukan karena ada kategori, dan recall PE diperlukan karena accuracy saja dapat menutupi kasus positif yang terlewat.

Latihan

Sebuah model memberikan accuracy tinggi tetapi seluruh kasus yang penting justru terlewat. Tahap apa yang perlu ditinjau? Mulailah dari evaluation untuk melihat jenis kesalahan, lalu kembali ke tujuan, distribusi data, atau modeling sesuai penyebabnya. CRISP-DM tidak mengharuskan proses berhenti setelah satu kali training.

Menjalankan cara berpikir CRISP DM



Gambar 2 Enam tahap saling memberi umpan balik. Panah evaluasi ke pemahaman data menunjukkan satu contoh iterasi.

Mulai dengan satu kalimat masalah

Untuk kasus pembelian, rumusan latihan adalah mengklasifikasikan Purchased dari Age dan EstimatedSalary. Pengguna yang dibayangkan adalah analis yang ingin mempelajari pola respons. Target yang diprediksi, unit baris dan waktu tersedianya fitur harus ditetapkan sebelum memilih algoritma.

Ubah setiap tahap menjadi bukti yang dapat diperiksa

Business understanding menghasilkan rumusan masalah dan kriteria penilaian. Data understanding menghasilkan daftar kolom, arti label, distribusi dan masalah kualitas. Data preparation menghasilkan X dan y yang terpisah serta aturan transformasi. Modeling menghasilkan baseline dan kandidat model. Evaluation menghasilkan tabel kesalahan yang dapat dijelaskan. Deployment menghasilkan cara memakai model beserta batas penggunaannya.

Jika pada evaluasi ditemukan accuracy tinggi tetapi seluruh PE terlewat, kita tidak langsung menambah model yang lebih rumit. Pertama, periksa distribusi kelas dan confusion matrix. Kemudian tinjau apakah metrik sudah sesuai tujuan. Perubahan dapat terjadi pada tahap evaluasi, pemodelan, atau bahkan definisi masalah.

Latihan menerapkan kerangka

Bayangkan Anda membuat pengarah pesan layanan kampus. Pada tahap pemahaman data, Anda menemukan label “keuangan” dipakai untuk pertanyaan biaya kuliah dan keluhan login pembayaran. Apa yang perlu dilakukan? Perjelas definisi label dan cara memberi label sebelum training; satu pesan yang sama seharusnya tidak berubah kelas hanya karena petugas penandanya berbeda.

Pada tahap deployment, model akan menerima pesan singkat dengan singkatan baru. Karena itu, evaluasi perlu mencerminkan bentuk pesan yang akan digunakan. Model dengan skor tinggi pada kumpulan pesan yang sangat berbeda belum menjawab kebutuhan aplikasi.

Hubungkan ke notebook 05–08. Tuliskan satu hasil konkret dari setiap tahap CRISP-DM berdasarkan berkas studi kasus yang tersedia.

Python untuk membaca dan memahami data

Python menyediakan library yang memudahkan pengolahan data. NumPy digunakan untuk array numerik, pandas untuk tabel, Matplotlib untuk grafik, dan scikit-learn untuk preprocessing serta model. Notebook 01 menggunakan library ini secara bertahap.

Array dan operasi per elemen

```
import numpy as np
tinggi = np.array([1.60, 1.70, 1.75])
berat = np.array([50, 65, 72])
indeks = berat / tinggi**2
print(indeks)
```

Operasi pembagian berlangsung pada pasangan elemen dengan posisi yang sama. Array tinggi dan berat masing-masing berisi tiga angka. Makna operasi tetap bergantung pada satuan; mengganti meter menjadi sentimeter tanpa menyesuaikan rumus akan mengubah hasil.

DataFrame dan struktur tabel

```
import pandas as pd
df = pd.read_csv('Social_Network_Ads.csv')
print(df.shape)
print(df.head())
print(df.dtypes)
print(df.isna().sum())
```

Kode ringkas di atas mengasumsikan file berada di direktori kerja. Notebook pada paket menggunakan jalur folder data yang sesuai. shape menghasilkan jumlah baris dan kolom. head hanya menampilkan lima baris awal; dtypes membantu memeriksa tipe data, sedangkan isna().sum() menghitung nilai kosong per kolom.

Memilih fitur dan target

```
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
print(X.shape)
print(y.value_counts())
```

Dua pasang kurung siku membentuk DataFrame dengan dua kolom fitur. Satu pasang kurung siku menghasilkan Series untuk target. Pada dataset pembelian terdapat 400 baris; penggunaan head pada tahap pengamatan tidak mengurangi 400 baris yang dipakai saat split.

Membaca statistik dengan konteks

describe merangkum jumlah data, rerata, simpangan baku, nilai minimum, kuartil dan maksimum. Ringkasan ini membantu menemukan nilai yang perlu diperiksa. Nilai ekstrem tidak otomatis salah; kita perlu memahami proses pengukuran dan konteks sebelum menghapusnya.

Latihan: apa perbedaan df.head() dan df = df.head()? Perintah pertama menampilkan bagian awal, sedangkan perintah kedua mengganti variabel df dengan hanya bagian awalnya. Kesalahan kecil ini dapat membuat proses training tanpa sengaja memakai data yang sangat sedikit.

Membaca data sebelum mengubahnya

Dataset adalah hasil pencatatan suatu proses. Satu baris dapat berarti satu pelanggan, satu kunjungan, satu pesan, atau satu gambar. Arti baris menentukan cara menghitung jumlah sampel dan cara memisahkan data. Dua baris tidak selalu berarti dua orang yang berbeda.

Langkah 1 Kenali jenis atribut

Jenis	Contoh	Perlakuan yang masuk akal
Numerik	Age, EstimatedSalary	Periksa satuan dan rentang; statistik numerik
Nominal	Country	Kategori tanpa urutan; one hot
Ordinal	Rendah, sedang, tinggi	Urutan bermakna; jarak antar tingkat belum tentu sama
Pengenal	NO atau ID	Penelusuran data; umumnya bukan prediktor
Target	Purchased atau label PE	Dipelajari sebagai jawaban; tidak masuk fitur

Langkah 2 Baca statistik dengan pertanyaan

Mean adalah jumlah nilai dibagi banyaknya nilai. Median adalah nilai tengah sesudah pengurutan. Modus adalah nilai yang paling sering muncul. Untuk nilai 20, 21, 22, 23 dan 100, mean adalah 37,2 sedangkan median 22. Perbedaan tersebut memberi petunjuk bahwa satu nilai besar menarik mean ke atas; belum membuktikan nilai 100 salah.

```
df[['Age', 'EstimatedSalary']].describe()
df['Purchased'].value_counts(dropna=False)
df.isna().sum()
df.duplicated().sum()
```

describe membantu membaca posisi dan sebaran angka. value_counts menunjukkan jumlah tiap label. isna menghitung data kosong; duplicated mencari baris identik. Baris identik perlu ditelaah karena dapat merupakan pengulangan pencatatan atau kejadian berbeda dengan atribut yang sama.

Langkah 3 Periksa hubungan dengan grafik

Histogram memperlihatkan sebaran satu fitur; diagram batang membandingkan jumlah kategori; scatter plot memperlihatkan pasangan dua angka. Titik yang berdekatan belum berarti identik. Hubungan pada grafik juga tidak membuktikan bahwa satu atribut menyebabkan atribut lain.

Kesalahan yang sering terjadi

Kolom berisi angka belum tentu numerik bermakna, misalnya nomor pelanggan. Nilai kosong juga tidak sama dengan nol. Mengganti seluruh nilai kosong dengan nol dapat menciptakan pola yang tidak sesuai proses pencatatan. Gunakan arti atribut untuk memutuskan perlakuan.

Hubungkan ke notebook 01. Baca shape, describe dan grafik sebelum memilih fitur. Pada notebook 05, ulangi pertanyaan yang sama untuk data Excel.

Input space dan feature space

Input space adalah ruang data masukan sebagaimana direpresentasikan sebelum transformasi yang sedang dibahas. Feature space adalah ruang koordinat yang digunakan untuk mempelajari pola. Transformasi dapat mempertahankan, menggabungkan atau memperluas atribut. Dalam contoh sederhana, kategori negara diubah menjadi beberapa indikator numerik.

Contoh one hot encoding

Negara	Usia	France	Germany	Spain
France	44	1	0	0
Spain	27	0	0	1
Germany	30	0	1	0

Negara merupakan kategori nominal. Memberi kode France = 0, Germany = 1 dan Spain = 2 dapat membuat algoritma berbasis jarak membaca urutan dan selisih yang tidak memiliki makna. One hot menyatakan keanggotaan kategori melalui indikator 0 atau 1.

```
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder(sparse_output=False)
country_features = encoder.fit_transform(df[['Country']])
print(encoder.get_feature_names_out())
```

fit_transform mempelajari kategori kemudian membentuk matriks indikator. Setelah encoder dilatih, gunakan transform pada data baru agar jumlah dan urutan kolom konsisten. Nama fitur memungkinkan kita menelusuri arti setiap koordinat.

Jumlah sampel dan dimensi fitur

Jika terdapat n sampel dan d fitur, matriks tabular X memiliki bentuk n × d. Setiap baris adalah satu sampel; setiap kolom adalah satu koordinat fitur. Ketika tiga atribut asal berupa Country, Age dan Salary diubah menjadi tiga indikator negara ditambah dua angka, dimensi berubah dari tiga menjadi lima kolom.

Transformasi representasi tidak selalu menambah informasi. One hot membuat kategori dapat diproses algoritma, tetapi tidak menciptakan atribut baru tentang orang tersebut. Sebaliknya, fitur turunan seperti rasio atau ringkasan dapat menyajikan hubungan atribut dengan cara yang lebih sesuai untuk tugas tertentu.

Makna tetap diperlukan

Mesin memproses angka sesuai algoritmanya. Namun kita tetap perlu mengetahui apa arti kolom, satuan angka, cara pengukuran dan waktu ketersediaannya. Dua matriks yang memiliki ukuran sama belum tentu dapat dipertukarkan. Kolom usia dan bobot istilah memiliki makna yang berbeda.

Latihan: sebuah kategori memiliki empat kemungkinan nilai dan dua fitur lain berupa angka. Jika seluruh kategori di-one-hot tanpa drop, jumlah fitur akhirnya adalah enam. Jumlah baris tidak berubah selama tidak ada sampel yang dihapus.

Melihat perubahan dari atribut menjadi koordinat



Gambar 3 Tiga kategori nominal menjadi tiga koordinat indikator. Urutan kolom dipertahankan setelah fit.

Langkah 1 Bedakan objek dengan representasinya

Country adalah atribut pada data asal. Sesudah one hot, Country_France, Country_Germany dan Country_Spain menjadi tiga fitur. Satu orang tetap satu sampel walaupun kolomnya bertambah. Karena itu, jumlah baris matriks mengikuti jumlah objek, sedangkan jumlah kolom mengikuti cara kita merepresentasikan objek.

Langkah 2 Hitung dimensi keluaran

Jika X memiliki Country, Age dan Salary, maka input terdiri atas tiga atribut. Dengan tiga kategori negara yang semuanya muncul pada train, hasil tanpa drop memiliki lima fitur: dua numerik dan tiga indikator. Pada ColumnTransformer di notebook 03, fitur numerik diletakkan lebih dahulu karena transformer num ditulis sebelum cat.

```
cols = preprocessor.get_feature_names_out()
print(cols)
print(X_train_array.shape)
print(X_test_array.shape)
```

Nama fitur adalah keterangan bagi setiap kolom. Jangan menebak urutan dari tabel mentah atau menulis ulang encoding di aplikasi. Urutan yang keliru dapat membuat angka penghasilan dibaca sebagai usia atau indikator negara ditukar artinya.

Langkah 3 Uji kategori yang belum dikenal

Notebook 03 memasukkan Country = Indonesia setelah encoder belajar dari kategori negara pada train. Dengan handle_unknown = ignore, semua indikator negara yang dikenal bernilai nol untuk kategori baru itu. Program tetap berjalan, tetapi model tidak otomatis mengetahui ciri Indonesia. Perlakuan ini harus dipahami saat membaca prediksi.

Input space dan feature space tidak selalu terpisah secara mutlak. Untuk data yang sejak awal berupa angka, sebagian koordinat dapat langsung menjadi fitur. Yang perlu dipahami adalah transformasi apa yang dilakukan dan informasi apa yang tetap atau berubah.

Hubungkan ke notebook 02 dan 03. Bandingkan bentuk data asal dengan nama fitur keluaran. Jelaskan mengapa kolom bertambah tetapi jumlah sampel tidak.

Teks dan citra sebagai representasi numerik

Teks menjadi bobot istilah

Kalimat merupakan urutan simbol. Untuk model tabular sederhana, teks dapat diubah menjadi jumlah atau bobot istilah. TF-IDF menggabungkan frekuensi istilah di dalam dokumen dengan informasi seberapa umum istilah tersebut pada kumpulan dokumen. Istilah yang muncul di hampir semua dokumen biasanya memberi daya pembeda yang lebih kecil.

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(max_features=300)
X_text = vectorizer.fit_transform(documents)
print(X_text.shape)
print(vectorizer.get_feature_names_out()[:10])
```

Setiap baris X_text merepresentasikan satu dokumen; setiap kolom merepresentasikan istilah dalam kosakata. Banyak elemen bernilai nol karena satu dokumen hanya memuat sebagian kosakata, sehingga representasi sparse menghemat penyimpanan. max_features membatasi ukuran kosakata, bukan jumlah dokumen.

Notebook 02 memakai dataset teks asli untuk demonstrasi representasi. Ketika representasi dipakai dalam klasifikasi, kosakata dan IDF harus dipelajari hanya dari train. Notebook pengayaan 09 memperlihatkan TF-IDF dan Naive Bayes dalam satu pipeline.

Citra menjadi piksel dan kanal

Citra grayscale dapat dipandang sebagai susunan intensitas berukuran tinggi × lebar. Citra RGB memiliki tiga kanal warna, sehingga bentuknya tinggi × lebar × 3. Pada gambar berukuran 16 × 16 RGB, terdapat 768 nilai kanal. Flatten meratakan nilai tersebut menjadi vektor panjang 768.

```
from PIL import Image
image = Image.open('fruits.png').convert('RGB')
small = np.asarray(image.resize((16, 16)))
vector = small.reshape(-1)
print(small.shape, vector.shape)
```

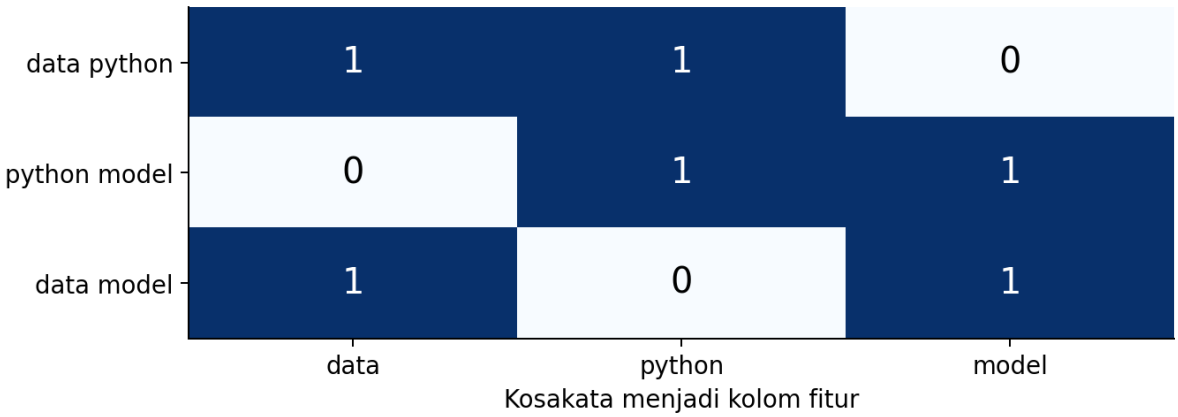
Notebook memuat gambar asli secara penuh, kemudian membuat versi kecil untuk memperlihatkan bentuk numeriknya. Resize mengurangi detail; flatten mengubah susunan penyajian. CNN mempertahankan dan memanfaatkan hubungan spasial piksel, sehingga tidak perlu selalu memperlakukan gambar sebagai tabel datar.

Kesamaan dan perbedaannya

Modalitas	Koordinat numerik contoh	Struktur yang penting
Tabular	Usia, gaji, indikator kategori	Arti dan satuan atribut
Teks	Bobot istilah TF-IDF	Kosakata dan konteks bahasa
Citra	Intensitas piksel atau fitur visual	Posisi spasial dan kanal

Ketiganya dapat direpresentasikan dengan angka, tetapi struktur, makna dan pilihan model tidak menjadi identik. Pilihan representasi adalah bagian dari pemecahan masalah.

Dari kalimat menjadi matriks istilah



Gambar 4 Contoh buatan tiga dokumen dengan hitungan kata. Ini bag of words sederhana, bukan output dataset sentimen.

Langkah 1 Bentuk kosakata

Kalimat “data python”, “python model” dan “data model” memiliki kosakata data, python dan model. Setiap dokumen menjadi satu baris; setiap kata menjadi satu kolom. Angka 1 pada gambar berarti kata muncul sekali. Jika kata tidak muncul, nilainya 0. Untuk kalimat lebih panjang, hitungan dapat melebihi 1.

Langkah 2 Bedakan hitungan kata dan TF IDF

Bag of words menyimpan hitungan. TF-IDF memberi bobot berdasarkan frekuensi kata dalam dokumen serta kelangkaannya di kumpulan dokumen. Secara intuitif, kata yang hadir hampir di semua dokumen cenderung kurang membedakan dokumen. Rumus IDF dengan smoothing yang umum pada TfidfVectorizer adalah log((1 + n) / (1 + df)) + 1; n adalah jumlah dokumen dan df jumlah dokumen yang memuat kata.

Pada contoh gambar, setiap kata muncul pada dua dari tiga dokumen, sehingga nilai IDF ketiganya sama. Nilai akhir TfidfVectorizer masih dipengaruhi normalisasi vektor. Karena itu, jangan mengharapkan keluaran TF-IDF persis 0 atau 1 seperti gambar hitungan kata.

Langkah 3 Pertahankan makna saat membersihkan teks

Preprocessing teks dapat mencakup tokenisasi, penghilangan kata tertentu dan stemming. Langkah tersebut merupakan pilihan, bukan kewajiban untuk semua tugas. Menghapus kata “tidak” pada “tidak bagus” dapat membalik informasi sentimen. Stemming juga dapat membantu penggabungan variasi kata, tetapi perlu dinilai terhadap tugas dan data.

Notebook 02 mengubah teks menjadi hitungan dan bobot istilah, lalu membandingkan bentuk matriksnya. Notebook 09 menggunakan TF-IDF dan Naive Bayes sebagai pengayaan klasifikasi yang ringan. Kosakata dan IDF pada klasifikasi dipelajari dari train saja.

Hubungkan ke notebook 02. Amati shape matriks sparse dan nama istilah. max_features = 300 membatasi kolom kosakata, bukan jumlah dokumen.

Dari gambar menjadi array numerik



Gambar 5 fruits.png dari bahan praktikum, versi kecilnya, dan satu kanal warna. Gambar kecil digunakan untuk mengamati bentuk array.

Langkah 1 Baca piksel dan kanal

Citra RGB memiliki tinggi, lebar dan tiga kanal: merah, hijau dan biru. Satu piksel dapat dinyatakan sebagai tiga angka intensitas. Dalam array uint8, setiap nilai kanal berada pada rentang 0 sampai 255. Warna yang terlihat berasal dari kombinasi ketiganya.

Langkah 2 Periksa bentuk sebelum menghitung

```
image = Image.open('fruits.png').convert('RGB')
small = np.asarray(image.resize((16, 16)))
print(small.shape)      # (16, 16, 3)
print(small[0, 0])      # tiga nilai kanal piksel kiri atas
vector = small.reshape(-1)
print(vector.shape)     # (768,)
```

Ada 16 × 16 × 3 = 768 nilai kanal. reshape mengubah susunan penyajian tanpa menambah isi piksel. Resize berbeda: proses ini menghasilkan ukuran gambar baru dan dapat mengurangi detail. Jika 100 gambar memiliki ukuran sama, flatten seluruh gambar dapat menghasilkan matriks 100 × 768.

Langkah 3 Hubungkan representasi dengan model

Model tabular dapat menerima vektor piksel, tetapi susunan spasial tidak lagi eksplisit dalam bentuk baris panjang. CNN menggunakan operasi pada lingkungan piksel untuk memanfaatkan pola lokal. Pelatihan CNN dapat dipelajari setelah dasar representasi, klasifikasi dan evaluasi dipahami.

Kesamaan tabular, teks dan gambar adalah kemampuannya direpresentasikan secara numerik. Perbedaannya tetap penting: angka pada tabel mempunyai nama atribut, angka teks berkaitan dengan kosakata, dan angka citra mempunyai posisi serta kanal. Gambar sering diproses sebagai tensor, bukan selalu matriks dua dimensi.

Hubungkan ke notebook 02. Bandingkan shape gambar, vektor, dan matriks teks. Sebutkan arti satu baris atau satu sumbu pada masing-masing representasi.

Preprocessing yang dapat dipercaya

Data awal sering memiliki nilai kosong, format tidak konsisten dan kategori yang belum berbentuk numerik. Preprocessing menyiapkan representasi yang konsisten. Langkahnya perlu menjaga pemisahan antara data yang digunakan untuk belajar dan data yang digunakan untuk menguji.

Pembersihan dan pembelajaran parameter

Menghapus spasi di ujung teks atau mengambil angka dari “23 TH” adalah aturan deterministik. Mengisi nilai kosong dengan median berbeda: median dihitung dari kumpulan data. Karena itu, statistik imputasi, parameter scaling dan kategori encoder dipelajari dari train; test hanya menggunakan aturan yang sudah terbentuk.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.25, stratify=y, random_state=42
)
```

test_size menentukan bagian data uji. stratify menjaga proporsi kelas sedekat mungkin dengan data asal. random_state membuat pembagian dapat diulang pada data yang sama. Pembagian acak belum menjamin independensi pasien, pengguna atau waktu; untuk data berulang diperlukan desain split yang sesuai.

Contoh imputasi

Misalkan usia train adalah 18, 20, kosong dan 22. Median dari nilai yang tersedia adalah 20, sehingga nilai kosong diisi 20. Jika test memiliki usia 60, angka itu tidak mengubah median train. Menghitung median dari gabungan train dan test memberi preprocessing akses pada informasi test.

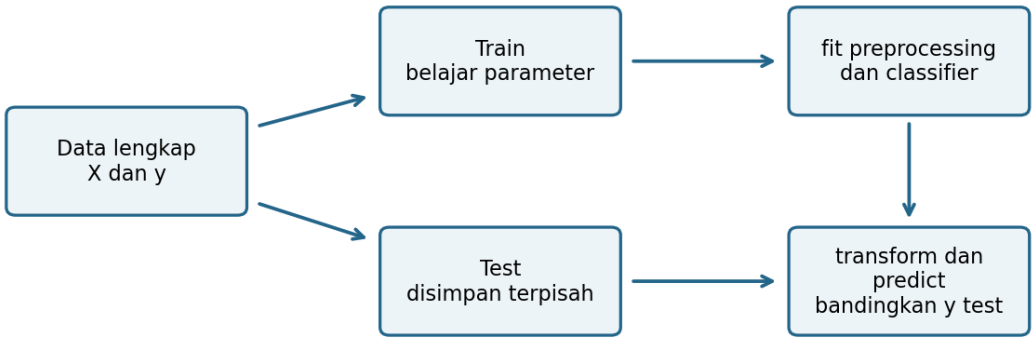
Pipeline numerik dan kategorikal

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
num = Pipeline([
  ('imputer', SimpleImputer(strategy='median')),
  ('scaler', StandardScaler())
])
cat = Pipeline([
  ('imputer', SimpleImputer(strategy='most_frequent')),
  ('encoder', OneHotEncoder(handle_unknown='ignore'))
])
```

ColumnTransformer menerapkan num hanya pada kolom numerik dan cat pada kolom kategorikal. Pipeline menyambungkan preprocessing dengan classifier. Saat pipeline.fit dipanggil pada train, seluruh tahap belajar dari train yang sama. Saat predict dipanggil, transformasi yang tersimpan diterapkan sebelum prediksi.

StandardScaler menghitung z = (x − rerata train) / simpangan baku train. Scaling relevan untuk k-NN karena perhitungan jarak peka terhadap satuan. Pohon keputusan tidak membutuhkan scaling dengan alasan yang sama, karena membagi data melalui ambang pada masing-masing fitur.

Memahami fit dan transform tanpa kebocoran



Gambar 6 Data uji mengikuti transformasi yang dipelajari dari data latih. Panah dari model ke pengujian membawa parameter yang sudah terbentuk.

Langkah 1 Pisahkan baris sebelum belajar statistik

Notebook 03 membagi Data.csv menjadi tujuh baris train dan tiga baris test. Dataset ini hanya demonstrasi mekanisme preprocessing. Notebook 04 memakai 400 baris pembelian dengan pembagian 300 train dan 100 test. Perbedaan proporsi ini disengaja; gunakan nilai split pada notebook yang sedang dijalankan.

Langkah 2 Pelajari parameter dari train

fit pada imputer menyimpan median atau modus. fit pada scaler menyimpan ukuran pusat dan skala. fit pada encoder menyimpan kategori. fit_transform menggabungkan kegiatan mempelajari parameter dan menerapkannya pada data yang sama. Karena itu, fit_transform digunakan pada train.

Langkah 3 Terapkan aturan yang sama pada test

transform pada test menggunakan parameter tersimpan. Jika nilai usia test sangat besar, median train tidak ikut berubah. Memanggil fit_transform pada test justru membuat aturan baru, sehingga fitur train dan test dapat memiliki skala atau kolom yang berbeda.

```
Atr = prep.fit_transform(Xtr)
Ate = prep.transform(Xte)
# Untuk pipeline yang juga berisi classifier:
pipe.fit(Xtr, ytr)
pred = pipe.predict(Xte)
```

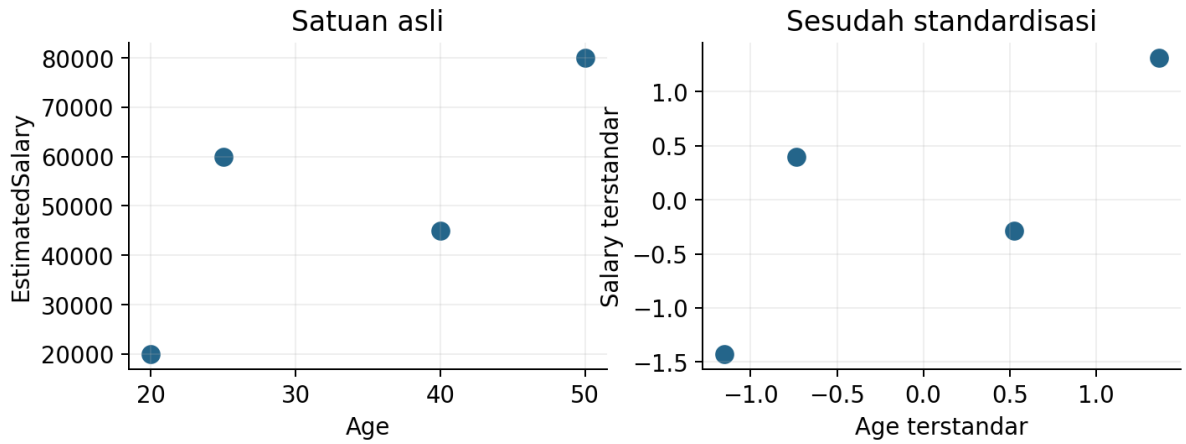
Pipeline menjalankan transformasi sebelum classifier secara otomatis. Saat fit, preprocessing dan classifier belajar pada train. Saat predict, preprocessing hanya mentransformasikan input dan classifier menghasilkan keluaran. ColumnTransformer mengatur kolom mana yang diproses oleh setiap cabang.

Pembersihan deterministik dan statistik belajar

Menghapus spasi atau mengubah “23 TH” menjadi 23 dapat didefinisikan sebagai aturan format tetap. Median dan kosakata harus dipelajari dari train. Jika aturan pembersihan dipilih setelah berulang kali melihat hasil test, test tetap dapat memengaruhi keputusan pengembangan secara tidak langsung.

Hubungkan ke notebook 03. Ubah Salary pada sampel baru lalu panggil transform. Periksa bahwa statistics_ milik imputer tidak berubah.

Mengapa skala mengubah arti jarak



Gambar 7 Empat sampel buatan sebelum dan sesudah StandardScaler. Koordinat berubah, identitas sampel tetap.

Langkah 1 Perhatikan besarnya angka

Usia dapat berbeda 10 tahun, sedangkan penghasilan berbeda 50.000 satuan. Jika jarak dihitung langsung, kuadrat selisih penghasilan jauh lebih besar. Akibatnya, kemiripan dapat didominasi satuan penghasilan. Dominasi tersebut belum tentu sesuai tujuan pembelajaran.

Langkah 2 Standardisasi dengan parameter train

StandardScaler menggunakan z = (x − μ_train) / σ_train. Jika mean usia train 30 dan simpangan bakunya 10, usia 40 menjadi z = 1. Artinya nilai itu satu simpangan baku di atas mean train. Angka negatif berarti di bawah mean; bukan data yang salah.

Standardisasi tidak otomatis membuat distribusi menjadi normal dan tidak mensyaratkan setiap fitur berdistribusi normal untuk dapat dihitung. Metode ini juga masih peka terhadap nilai ekstrem. Pada fitur konstan, implementasi menangani skala agar pembagian tetap terdefinisi.

Langkah 3 Sesuaikan dengan algoritma

k-NN pada notebook 04 menggunakan jarak, sehingga scaling merupakan bagian penting pipeline. Decision tree pada studi kasus memakai pemisahan nilai menurut ambang; pipeline workshop tidak menambahkan scaling pada fitur numeriknya. Preprocessing tidak harus identik untuk semua algoritma.

Pilih metode dengan alasan

MinMaxScaler memetakan berdasarkan minimum dan maksimum train; data baru dapat berada di luar rentang hasil train. RobustScaler menggunakan median dan rentang antarkuartil. Ketiganya mengubah skala, tetapi tidak membenarkan penghapusan outlier secara otomatis. Nilai ekstrem yang valid tetap dapat membawa informasi.

Hubungkan ke notebook 03 dan 04. Cari scaler di dalam pipeline. Jelaskan mengapa mean atau simpangan baku tidak dihitung ulang ketika sampel baru diprediksi.

Klasifikasi menggunakan k NN

k-Nearest Neighbors mengklasifikasikan masukan berdasarkan label tetangga terdekat. Pada k = 5, model mengambil lima contoh latih terdekat dan melakukan voting. Sebelum menghitung jarak, notebook 04 mengisi nilai numerik kosong dengan median train dan menstandardisasi Age serta EstimatedSalary.

Langkah 1 Pisahkan data dan bangun pipeline

Dataset memiliki 400 baris. Split terstratifikasi dengan test_size = 0,25 dan random_state = 42 menghasilkan 300 train dan 100 test. Purchased menjadi target. User ID tidak digunakan sebagai fitur karena nomor identitas bukan besaran perilaku.

```
def make_knn(k=5, with_scaling=True):
  steps = [('imputer', SimpleImputer(strategy='median'))]
  if with_scaling:
    steps.append(('scaler', StandardScaler()))
  steps.append(('knn', KNeighborsClassifier(n_neighbors=k)))
  return Pipeline(steps)
```

Langkah 2 Bandingkan kandidat pada train

Notebook membandingkan tanpa scaling dan dengan scaling melalui 5-fold stratified CV pada train. Selanjutnya, k diuji pada kandidat 1, 3, 5, 7, 9, 11, 15, 21 dan 31. Pemilihan menggunakan mean accuracy validasi. Jika rata-ratanya sama, notebook memilih k yang lebih kecil.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(
  make_knn(k=7), X_train, y_train, cv=cv, scoring='accuracy'
)
```

Setiap fold mempelajari imputasi dan scaling dari bagian latihan fold tersebut. Holdout test belum dipakai untuk memilih k. Pada hasil paket, k terpilih adalah 7; nilai ini bukan aturan universal untuk semua dataset.

Langkah 3 Latih model terpilih dan baca hasil test

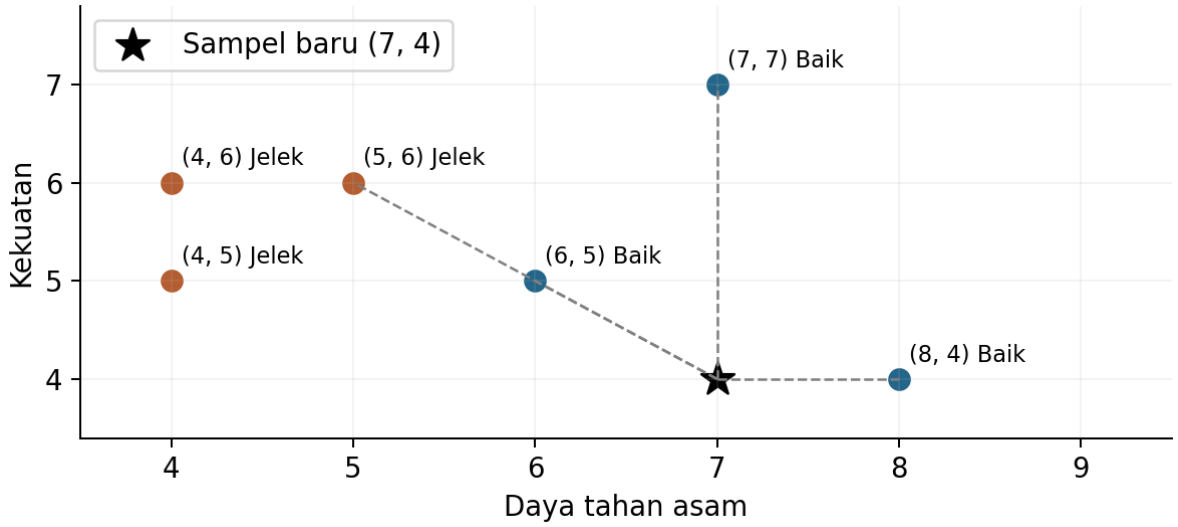
```
final_model = make_knn(k=best_k)
final_model.fit(X_train, y_train)
pred_test = final_model.predict(X_test)
```

Laporan tersimpan menunjukkan accuracy 0,92 pada 100 baris test. Untuk Purchased = 1, precision dan recall masing-masing sekitar 0,889. Bandingkan dengan baseline mayoritas dan baca confusion matrix agar memahami kelas mana yang masih salah diprediksi.

Langkah 4 Simpan hasil untuk ditelusuri

Notebook menyimpan knn_pipeline_final.joblib, knn_prediksi_test.csv dan knn_evaluasi.txt pada praktikum/hasil/modul04. Model tersimpan mencakup preprocessing dan classifier. Prediksi baru harus memakai nama dan arti fitur yang sama.

Menghitung jarak dan voting k NN



Gambar 8 Contoh kualitas tisu diadaptasi dari Klasifikasi Tabular dengan K-NN halaman 10–12. Garis menunjukkan empat tetangga terdekat.

Sampel baru q = (7, 4) memiliki dua atribut. Untuk memahami algoritma, gunakan jarak Euclidean pada angka asli seperti contoh slide. Contoh ini demonstrasi hitung; pada data dengan satuan yang sangat berbeda, pertimbangkan scaling seperti praktikum pembelian.

Langkah 1 Hitung jarak setiap sampel

$d(x, q) = \sqrt{(x_1 - 7)^2 + (x_2 - 4)^2}$. Untuk (6, 5), jaraknya $\sqrt{(-1)^2 + 1^2} = \sqrt{2} \approx 1{,}414$. Jarak selalu tidak negatif; semakin kecil berarti semakin dekat pada representasi yang dipilih.

Sampel	Kelas	Jarak ke q	Masuk k sama dengan 4
(8, 4)	Baik	1,000	Ya
(6, 5)	Baik	1,414	Ya
(5, 6)	Jelek	2,828	Ya
(7, 7)	Baik	3,000	Ya
(4, 5)	Jelek	3,162	Tidak
(4, 6)	Jelek	3,606	Tidak

Langkah 2 Urutkan dan lakukan voting

Empat jarak terkecil memberikan tiga suara Baik dan satu Jelek. Prediksi contoh adalah Baik. Label sampel baru ditentukan oleh tetangga yang dipilih, bukan oleh seluruh data secara langsung. Pada notebook 04, k = 5 digunakan untuk demonstrasi awal. Nilai k model final dipilih melalui cross-validation setelah imputasi serta standardisasi.

Langkah 3 Pahami peran k

k kecil lebih peka terhadap contoh lokal dan noise. k besar meratakan keputusan lebih luas dan dapat mengabaikan kelompok kecil. Nilai ganjil mengurangi peluang seri pada voting dua kelas, tetapi bukan jaminan umum untuk semua situasi. Tentukan aturan voting dan pemilihan k sebelum evaluasi akhir.

Hubungkan ke notebook 04. Jelaskan mengapa mengganti k berarti mengganti konfigurasi model. Hindari memilih k hanya berdasarkan skor test yang dilihat berulang kali.

Membaca evaluasi klasifikasi

Evaluasi menilai kemampuan model pada data yang tidak digunakan saat training. Accuracy saja belum selalu cukup, terutama ketika jumlah kelas tidak seimbang. Confusion matrix memperlihatkan jenis kesalahan secara langsung.

Aktual dan prediksi	Prediksi negatif	Prediksi positif
Aktual negatif	TN atau true negative	FP atau false positive
Aktual positif	FN atau false negative	TP atau true positive

TN dan TP merupakan prediksi benar. FP adalah alarm positif pada sampel yang sebenarnya negatif. FN adalah kasus positif yang terlewat. Dalam grafik scikit-learn yang digunakan pada notebook, label aktual berada pada baris dan label prediksi pada kolom.

Metrik	Rumus	Makna
Accuracy	(TP + TN) / N	Proporsi seluruh prediksi benar
Precision	TP / (TP + FP)	Ketepatan dari seluruh prediksi positif
Recall	TP / (TP + FN)	Proporsi positif yang berhasil ditemukan
F1	2PR / (P + R)	Rerata harmonik precision dan recall
Specificity	TN / (TN + FP)	Proporsi negatif yang dikenali

Contoh perhitungan

Pada model PE terpilih, test memiliki TP = 7, FN = 3, FP = 8 dan TN = 123. Recall adalah 7/10 = 0,70. Precision adalah 7/15 ≈ 0,467. Accuracy adalah 130/141 ≈ 0,922. Model menemukan tujuh dari sepuluh positif, tetapi delapan dari 15 prediksi positif merupakan false positive.

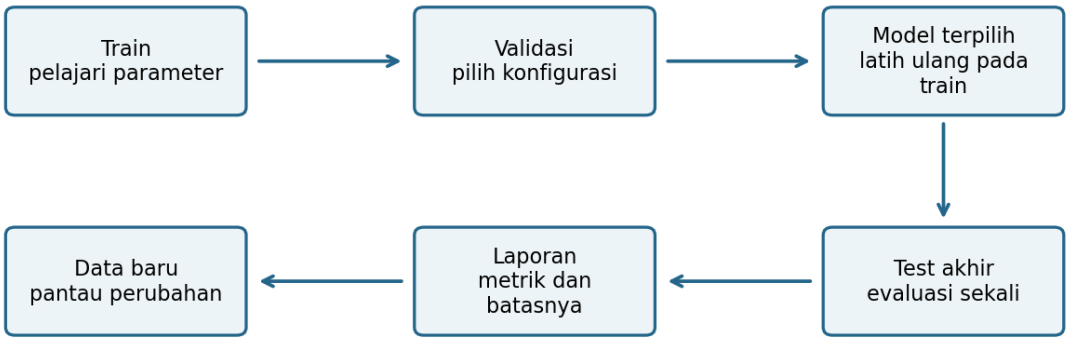
Jika semua 141 sampel diprediksi Non PE, 131 prediksi menjadi benar. Accuracy baseline sekitar 0,929, tetapi recall PE nol. Dengan demikian, accuracy yang lebih tinggi belum tentu berarti model lebih berguna untuk menemukan kelas positif.

Memilih ukuran sesuai tujuan

Precision relevan ketika banyak alarm salah menimbulkan pekerjaan tambahan. Recall relevan ketika melewatkan positif menjadi perhatian utama. Keduanya tetap perlu dibaca bersama. Rata-rata weighted memberi bobot lebih besar kepada kelas yang banyak, sehingga dapat menutupi kelemahan pada kelas minoritas.

Saat tidak ada prediksi positif, penyebut precision nol. Kode menggunakan zero_division = 0 agar pelaporan tetap konsisten. Angka ini harus dibaca bersama confusion matrix. Sebuah metrik adalah ringkasan perilaku pada kumpulan data tertentu, bukan jaminan kebenaran setiap prediksi individual.

Memisahkan pemilihan model dari penilaian akhir



Gambar 9 Alur pengembangan lanjutan. Data validasi dapat berasal dari sebagian train atau cross validation di dalam train.

Underfitting dan overfitting

Underfitting terjadi ketika model terlalu sederhana untuk menangkap pola yang relevan, sehingga kinerjanya pada train dan data baru kurang baik. Overfitting terjadi ketika model terlalu menyesuaikan detail train, termasuk noise, sehingga kinerjanya menurun pada data baru. Selisih skor train dan validasi dapat menjadi petunjuk, bukan diagnosis tunggal.

Parameter dan hyperparameter

Median imputasi dan ambang yang dipelajari pohon adalah hasil fit. Sebaliknya, n_neighbors, max_depth dan min_samples_leaf ditentukan sebelum fit dan disebut hyperparameter. Mengubahnya berarti mencoba konfigurasi berbeda. Keputusan memilih konfigurasi perlu menggunakan data validasi, bukan test akhir.

Mengapa melihat test berulang dapat menyesatkan

Jika Anda mencoba banyak konfigurasi lalu memilih yang paling tinggi pada test, pilihan tersebut sudah menyesuaikan diri dengan test. Skor test itu tidak lagi menjadi penilaian independen yang sama kuatnya. Gunakan validasi atau cross validation pada train untuk memilih, kemudian evaluasi model terpilih pada test yang disimpan.

Pada cross validation, train dibagi menjadi beberapa fold. Secara bergiliran, satu fold digunakan untuk validasi dan sisanya untuk fit. Preprocessing perlu ikut di dalam pipeline agar statistiknya juga dihitung ulang hanya dari bagian latihan setiap fold.

Validasi pada praktikum

Notebook 04 memilih k berdasarkan mean accuracy pada 5-fold CV train. Notebook 06 memilih konfigurasi pohon berdasarkan mean F1 kelas PE pada 5-fold CV train. Setelah pemilihan selesai, model dilatih pada seluruh train dan dinilai pada holdout test. Skor terbaik hanya berarti terbaik di antara kandidat dan protokol yang diuji.

Hubungkan ke notebook 04 dan 06. Amati tabel kandidat CV, metrik pemilihan dan konfigurasi terpilih. Simpan test akhir untuk penilaian setelah pilihan ditetapkan.

Memahami data studi kasus preeklamsia

Studi kasus menggunakan berkas dataKasus-1.xlsx, sheet 2022, berisi 561 baris dan 13 kolom. Berkas Excel asli dan CSV seluruh sheet disertakan pada studi_kasus/data/asli. Notebook 05 memulai proses dari berkas asli tersebut.

Kolom atau kelompok	Peran dalam latihan
NO dan NAMA	Pengenal; dipertahankan pada data asli, tidak dipakai sebagai fitur
USIA dan PARITAS	Fitur numerik
JARAK KELAHIRAN	Fitur kategorikal
RIW HIPERTENSI, OBESITAS, RIW DM	Fitur kategorikal
RIW HIPERTENSI/PE DALAM KELUARGA	Fitur kategorikal
SOSEK RENDAH	Fitur kategorikal sesuai kategori pada sumber
RIW PE	Tersedia untuk telaah; dikeluarkan dari model utama
PE/Non PE	Target klasifikasi
Unnamed: 12	Sebagian berisi catatan bebas; tidak dipakai sebagai fitur

Masalah format yang perlu dibersihkan

Usia tercatat dalam bentuk seperti “23 TH”, “22 th” dan angka langsung. Ada tiga nilai usia kosong. Jarak kelahiran memiliki spasi tidak konsisten dan tujuh nilai kosong. Pembersihan menyamakan format tanpa langsung mengisi data kosong sebelum split.

```
usia = raw['USIA'].astype('string')
angka = usia.str.extract(r'(\d+)', expand=False)
raw['USIA'] = pd.to_numeric(angka, errors='coerce')
```

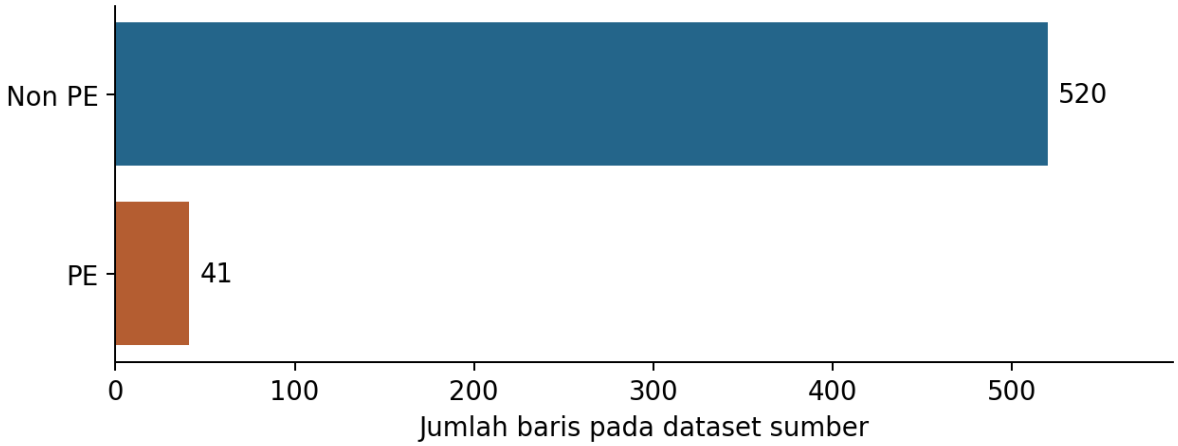
astype string menjaga nilai numerik tetap dapat diproses sebagai teks. Ekstraksi mengambil digit, kemudian to_numeric mengubahnya menjadi angka. errors = coerce menjadikan nilai yang tidak dapat dibaca sebagai missing value untuk diperiksa atau diimputasi sesuai aturan.

Definisi target dan waktu fitur

Setelah trim, label Non PE dipetakan menjadi 0. Label PE, PEB, Eklamsia, PE gemelli dan PEB impending eklampsi dipetakan menjadi 1 mengikuti pengelompokan pada notebook sumber. Hasilnya 520 Non PE dan 41 PE. Pengelompokan ini merupakan definisi operasional latihan yang masih memerlukan konfirmasi ahli untuk penelitian.

RIW PE memuat istilah kejadian klinis dan waktu pencatatannya belum jelas. Kolom tetap tersedia utuh, tetapi model utama memakai delapan fitur lain. Tugas latihan dinyatakan sebagai klasifikasi label pada data yang tersedia; klaim prediksi dini memerlukan verifikasi waktu fitur dan onset.

Menelaah dataset PE secara bertahap



Gambar 10 Jumlah kelas setelah pemetaan label pada 561 baris dataset asli. Positif PE berjumlah 41, sekitar 7,3 persen.

Langkah 1 Periksa label sebelum mengisi fitur

Label pada data sumber tidak hanya berupa PE dan Non PE. Fungsi load_data menghapus spasi dan memetakan Non PE menjadi 0; PE, PEB, Eklamsia, PE gemelli dan PEB impending eklampsi menjadi 1 untuk tugas biner workshop. Ini merupakan definisi pengelompokan untuk latihan, bukan pengganti penetapan atau penyamaan diagnosis klinis.

Label kosong atau label di luar pemetaan menyebabkan error agar tidak diam-diam dimasukkan ke kelas tertentu. Kesalahan pada target berbeda dari nilai fitur yang kosong: target adalah jawaban untuk melatih dan mengevaluasi model sehingga definisinya harus jelas.

Langkah 2 Pisahkan informasi penelusuran dari prediktor

NO dan NAMA tidak digunakan sebagai fitur. Nilai NAMA pada berkas berupa penanda seperti NAMA 1, tetapi kita belum memastikan independensi pasien. Kolom Unnamed: 12 memiliki beberapa catatan dan tidak digunakan. RIW PE juga dikeluarkan karena waktu dan makna pencatatannya perlu dikonfirmasi agar tidak membocorkan label saat ini.

Langkah 3 Bersihkan bentuk tanpa menghapus seluruh baris

USIA dapat berupa angka atau teks seperti “23 TH”. Parser mengambil angka lalu mengubahnya menjadi numerik. Pada sumber terdapat tiga USIA kosong dan tujuh JARAK KELAHIRAN kosong. Pengisian nilainya dilakukan dalam pipeline setelah split, sehingga seluruh 561 baris tetap tersedia untuk eksperimen.

Parser sederhana ini sesuai bentuk yang ditemukan pada bahan. Pada data baru, string seperti “20–25” tidak boleh langsung ditafsirkan menjadi satu usia tanpa penelaahan. Aturan yang berhasil pada satu file tetap perlu diuji terhadap bentuk input baru.

Hubungkan ke notebook 05. Bandingkan nilai unik label sebelum dan sesudah strip. Periksa bahwa X mempunyai delapan kolom dan y berisi 520 nilai 0 serta 41 nilai 1.

Membangun pipeline studi kasus

Notebook 06 membaca data_bersih.csv dari notebook 05. Data berisi fitur sebelum imputasi dan target_PE. Dengan split terstratifikasi 75/25 dan seed 42, terdapat 420 train dan 141 test. Train memiliki 31 PE; test memiliki 10 PE dan 131 Non PE.

Langkah 1 Transformasikan kolom sesuai jenisnya

USIA dan PARITAS menggunakan imputasi median. Enam fitur kategorikal menggunakan imputasi modus lalu one hot. Statistik dan kategori dipelajari hanya dari train. Hasil transformasi pada paket mempunyai 15 fitur numerik.

```
numeric = Pipeline([
  ('imputer', SimpleImputer(strategy='median'))
])
category = Pipeline([
  ('imputer', SimpleImputer(strategy='most_frequent')),
  ('onehot', OneHotEncoder(
    handle_unknown='ignore', sparse_output=False))
])
preprocess = ColumnTransformer([
  ('num', numeric, NUMERIC),
  ('cat', category, CATEGORICAL)
])
```

Langkah 2 Bentuk pohon awal sebagai pembanding

```
fixed_model = Pipeline([
  ('preprocess', preprocess),
  ('classifier', DecisionTreeClassifier(
    max_depth=3, min_samples_leaf=10,
    class_weight='balanced', random_state=42))
])
fixed_model.fit(X_train, y_train)
```

max_depth membatasi kedalaman dan min_samples_leaf membatasi jumlah sampel per daun. class_weight = balanced meningkatkan bobot relatif kelas yang sedikit. Ini adalah konfigurasi pohon awal, bukan otomatis model yang diekspor.

Langkah 3 Pelajari kesalahan pada validasi internal

Untuk demonstrasi awal, notebook membagi train lagi menjadi development dan validation dengan test_size = 0,25 dan random_state = 21. Baseline dan pohon dibandingkan pada validation internal ini. Holdout test tetap disimpan untuk tahap penilaian setelah pemilihan konfigurasi.

Pemilihan selanjutnya menggunakan cross-validation pada 420 baris train. Baseline, pohon awal dan pohon terpilih akhirnya dinilai pada test yang sama agar jenis kesalahan dapat dibandingkan secara jelas.

Memilih pohon melalui cross validation

Langkah 1 Tetapkan kandidat dan tujuan pemilihan

Notebook 06 mencoba tiga kedalaman, tiga ukuran minimum daun dan dua pilihan bobot kelas. Jumlah kandidat adalah 3 × 3 × 2 = 18. Metrik pemilihan adalah F1 kelas PE, sehingga precision dan recall positif ikut memengaruhi pemilihan.

```
param_grid = {
  'classifier__max_depth': [2, 3, 4],
  'classifier__min_samples_leaf': [5, 10, 20],
  'classifier__class_weight': [None, 'balanced']
}
cv = StratifiedKFold(
  n_splits=5, shuffle=True, random_state=42
)
search = GridSearchCV(
  make_tree(), param_grid, scoring='f1', cv=cv,
  refit=True, n_jobs=1, error_score='raise'
)
search.fit(X_train, y_train)
final_model = search.best_estimator_
```

Langkah 2 Baca keluaran CV dengan benar

best_params_ menyimpan konfigurasi terpilih; best_score_ adalah mean F1 validasi pada train. Pada paket, hasilnya max_depth = 2, min_samples_leaf = 5 dan class_weight = balanced, dengan mean CV F1 sekitar 0,454. Angka ini berbeda dari F1 holdout karena data yang dinilai berbeda.

Istilah mean_test_score di cv_results_ berarti skor bagian validasi pada setiap fold CV. Nama tersebut tidak menunjuk holdout test 141 baris. Pipeline ikut dalam pencarian agar imputer dan encoder hanya fit pada bagian latihan setiap fold.

Langkah 3 Gunakan model hasil refit

refit = True melatih konfigurasi terpilih pada seluruh 420 baris train. best_estimator_ sudah berupa pipeline terlatih. Test digunakan setelah tahap ini untuk mengukur kinerja, bukan untuk memilih ulang kedalaman atau ukuran daun.

Langkah 4 Simpan keputusan untuk notebook berikutnya

laporan_praktikum06.json mencatat best_params, split, fitur dan metrik. Notebook 07 membaca informasi ini untuk merekonstruksi dan memeriksa model yang akan diekspor. ringkasan_cv.csv menyimpan ringkasan validasi; metrik_perbandingan.csv mencatat hasil test ketiga model.

Dengan 31 positif pada train, masing-masing fold validasi hanya memuat sedikit PE. Variasi antar-fold perlu dibaca bersama mean; hasil pemilihan belum membuktikan ketahanan pada pasien, waktu atau tempat lain.

Mengikuti keputusan pohon dari fitur ke kelas

Decision tree membagi sampel melalui pertanyaan bertingkat. Pada setiap simpul, satu fitur dibandingkan dengan ambang. Sampel bergerak ke salah satu cabang dan akhirnya mencapai daun. Kelas keluaran berasal dari komposisi kelas pada daun sesuai mekanisme model.

Mengapa satu pemisahan dipilih

Secara intuitif, pohon mencari pembagian yang membuat kelompok anak lebih seragam kelasnya. Ukuran Gini untuk dua kelas adalah $1 - p_0^2 - p_1^2$. Kelompok yang seluruhnya satu kelas memiliki Gini 0. Kelompok dengan separuh kelas 0 dan separuh kelas 1 memiliki Gini 0,5. Pengurangan ketidakmurnian dibandingkan dengan mempertimbangkan besar kelompok anak.

Ini menjelaskan prinsip pemisahan, bukan penghitungan ulang pohon workshop. Pada model workshop, class_weight = balanced membuat perhitungan mempertimbangkan bobot kelas; angka proporsi berbobot dapat berbeda dari hitungan baris biasa.

Bagaimana fitur kategori dibaca pohon

Sesudah one hot, kategori menjadi indikator 0 atau 1. Pertanyaan pada indikator dengan ambang 0,5 berarti memisahkan keanggotaan kategori: nilai 0 masuk satu cabang, nilai 1 masuk cabang lain. Fitur numerik seperti usia tetap mempunyai ambang angka. Nama panjang pada aturan membantu melacak indikator ke atribut asalnya.

Membaca aturan model yang benar benar disimpan

```
names = final_model.named_steps['preprocess'].get_feature_names_out()
from sklearn.tree import export_text
print(export_text(
   final_model.named_steps['classifier'], feature_names=list(names)
))
```

Baca dari baris paling kiri, ikuti kondisi menuju baris yang semakin menjorok, lalu lihat kelas pada daun. Notebook 06 juga menampilkan gambar pohon. Untuk menelusuri satu input, gunakan hasil transformasi pipeline karena nama dan urutan fitur pada pohon mengikuti data sesudah preprocessing.

Membatasi kompleksitas dan menafsirkan aturan

Pada model awal, max_depth = 3 dan min_samples_leaf = 10. Hasil CV pada paket memilih max_depth = 2 dan min_samples_leaf = 5. Kedalaman membatasi tingkat pemisahan, sedangkan min_samples_leaf membatasi jumlah sampel pada daun. Keduanya membantu mengendalikan kompleksitas, tetapi tidak menjamin generalisasi. Aturan pohon menggambarkan pola pada dataset, bukan bukti bahwa suatu faktor menyebabkan kejadian PE.

Hubungkan ke notebook 06. Pilih satu jalur pada gambar pohon. Jelaskan setiap kondisi dengan nama fitur, lalu bedakan kesimpulan model dengan kesimpulan sebab akibat.

Menafsirkan hasil studi kasus

Ketiga model berikut dievaluasi pada 141 baris holdout test yang sama. Konfigurasi pohon terpilih ditentukan terlebih dahulu melalui CV train. Kelas positif adalah kelompok PE.

Ukuran	Baseline	Pohon awal	Pohon CV
Accuracy	0,929	0,823	0,922
Balanced accuracy	0,500	0,766	0,819
Precision PE	0,000	0,241	0,467
Recall PE	0,000	0,700	0,700
F1 PE	0,000	0,359	0,560
Specificity	1,000	0,832	0,939
TP dan FN	0 dan 10	7 dan 3	7 dan 3
FP dan TN	0 dan 131	22 dan 109	8 dan 123

Memahami perbedaan kesalahan

Baseline melewatkan seluruh PE. Kedua pohon menemukan tujuh PE dan melewatkan tiga. Pohon terpilih menghasilkan delapan false positive, dibandingkan 22 pada pohon awal. Recall sama, tetapi precision dan F1 pohon terpilih lebih tinggi pada test ini. Pemilihan model tetap berdasarkan CV train; tabel test merupakan pelaporan hasil sesudah pemilihan.

Balanced accuracy adalah rata-rata recall kedua kelas. Untuk model terpilih: (0,700 + 0,939) / 2 ≈ 0,819. Ukuran ini memberi bobot sama pada kedua kelas, berbeda dari accuracy yang menghitung seluruh baris secara langsung.

Membaca batas hasil

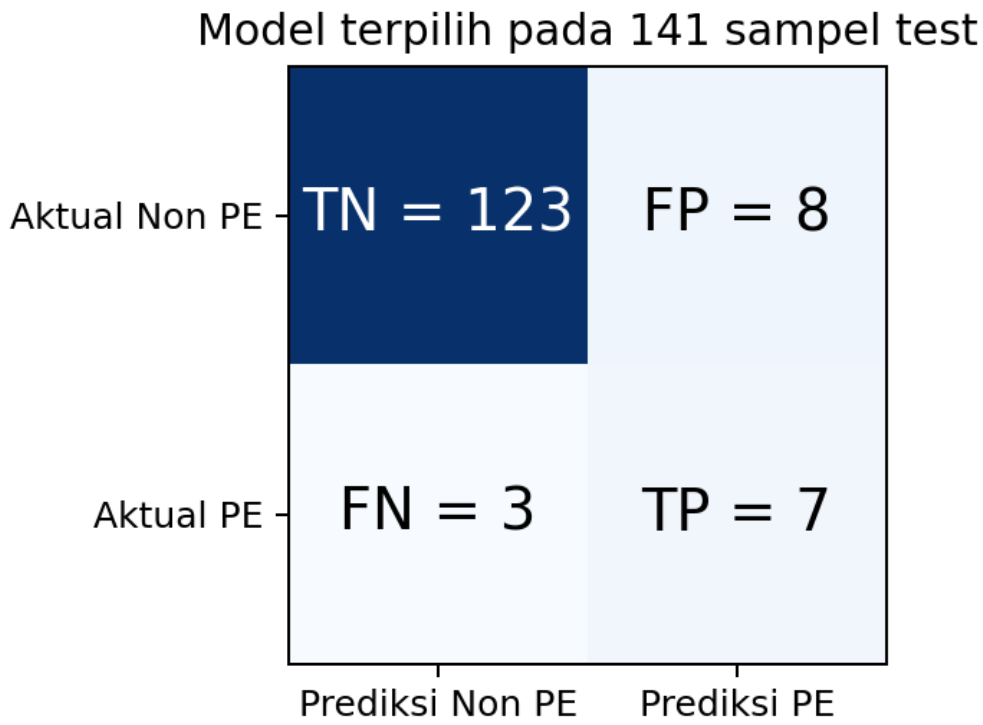
Hanya terdapat sepuluh PE pada test. Satu perubahan prediksi positif benar menjadi salah mengubah recall sebesar 0,10. Split dilakukan pada tingkat baris; independensi pasien dan waktu pencatatan belum terverifikasi. Hasil ini adalah latihan klasifikasi label, bukan validasi penggunaan klinis.

Menelusuri hasil eksperimen

Berkas pada studi_kasus/hasil	Isi
metrik_perbandingan.csv	Hasil test baseline, pohon awal dan pohon CV
laporan_praktikum06.json	Konfigurasi terpilih, split dan metrik
prediksi_test.csv	Prediksi dan skor model terpilih dari 06
prediksi_test_07.csv dan metrik_07.csv	Pemeriksaan reproduksi hasil saat ekspor
aturan_pohon_07.txt	Aturan model yang diekspor
manifest_praktikum07.csv	Daftar artefak beserta checksum

Indeks atau nomor baris pada laporan digunakan untuk menelusuri record, bukan sebagai bukti pengenal pasien unik. Cocokkan label aktual, prediksi dan skor antara notebook 06 dan 07 sebelum memakai model pada notebook 08.

Membaca empat jenis keluaran evaluasi



Gambar 11 Confusion matrix pohon terpilih melalui CV train. Baris adalah label aktual dan kolom adalah prediksi.

Langkah 1 Tetapkan arti kelas positif

Dalam studi kasus, positif berarti target PE = 1. True positive berjumlah 7: aktual PE dan diprediksi PE. False negative berjumlah 3: aktual PE tetapi diprediksi Non PE. False positive berjumlah 8: aktual Non PE tetapi diprediksi PE. True negative berjumlah 123.

Langkah 2 Hitung metrik dengan penyebut yang tepat

Accuracy = (123 + 7) / 141 = 0,922. Precision PE = 7 / (7 + 8) = 0,467: dari semua prediksi PE, sekitar 46,7 persen sesuai label aktual. Recall PE = 7 / (7 + 3) = 0,700: dari seluruh PE pada test, 70 persen ditemukan. Specificity = 123 / (123 + 8) = 0,939.

F1 = 2TP / (2TP + FP + FN) = 14 / 25 = 0,560. F1 menggabungkan precision dan recall, tetapi tidak memasukkan TN secara langsung. Dua model dapat memiliki F1 sama dengan jenis kesalahan yang berbeda; confusion matrix tetap perlu dibaca.

Langkah 3 Bandingkan dengan tebakan sederhana

Baseline selalu memilih Non PE. Karena terdapat 131 Non PE pada test, accuracy-nya 131/141 = 0,929, tetapi recall PE = 0. Untuk baseline ini tidak ada prediksi positif, sehingga precision secara matematis memiliki penyebut nol; laporan kode menggunakan zero_division = 0 agar ditampilkan sebagai 0.

Langkah 4 Nyatakan kesimpulan dengan batasnya

Pohon menemukan lebih banyak PE daripada baseline, sambil menghasilkan lebih banyak false positive. Ini adalah deskripsi hasil holdout baris, bukan bukti kelayakan klinis. Dengan hanya sepuluh positif pada test, perubahan satu hasil positif mengubah recall sebesar sepuluh poin persentase.

Hubungkan ke notebook 06 dan 07. Cocokkan angka gambar dengan baris Pohon terpilih via CV dalam metrik_perbandingan.csv dan prediksi_test.csv. Pastikan kelas positif dan arah sumbu sama sebelum menghitung.

Menyimpan model untuk inferensi

Model yang digunakan aplikasi harus menerima fitur dengan arti, nama dan urutan yang sesuai dengan proses pelatihan. Menyimpan classifier saja berisiko membuat preprocessing pada aplikasi berbeda. Karena itu, notebook 07 menyimpan satu objek pipeline yang berisi imputasi, encoding dan classifier.

Ekspor dan muat ulang

```
import joblib
joblib.dump(final_model, 'pe_workshop_pipeline.joblib')
loaded = joblib.load('pe_workshop_pipeline.joblib')
prediksi = loaded.predict(contoh_input)
skor = loaded.predict_proba(contoh_input)
```

Notebook 07 membaca best_params dari laporan_praktikum06.json, merekonstruksi split dan pipeline terpilih, lalu melatihnya pada train yang sama. Prediksi dan skor 141 baris test diperiksa terhadap hasil notebook 06 sebelum ekspor. Jalur pembentukan model final adalah notebook 06 lalu 07; train_export pada helper pe_workflow.py membentuk pohon awal dengan konfigurasi berbeda.

predict menghasilkan kelas. predict_proba menghasilkan skor per kelas; posisi kelas perlu dilihat melalui classes_. Skor pohon berasal dari informasi kelas pada daun dan dapat dipengaruhi class_weight. Skor ini belum otomatis merupakan risiko absolut individual yang terkalibrasi.

Model card

Model card mencatat jumlah data, daftar fitur, seed, ukuran split, metrik, versi scikit-learn dan batas penggunaan. Informasi ini membantu memastikan model yang dipanggil adalah artefak yang dimaksud. Artefak bawaan mencatat Python 3.10.18 dan scikit-learn 1.7.1. Gunakan environment sesuai requirements.txt. Untuk membangun ulang artefak, jalankan notebook 06–07, lalu gunakan model dan model card yang dihasilkan bersama.

Dua contoh deployment dalam paket

Aspek	Versi 1 asli	Versi 2 studi kasus
Model	XGBoost pada aplikasi contoh	Decision tree hasil workshop
Masukan	9 prediktor kategori	8 prediktor numerik dan kategori
Preprocessing	Kontrak 21 fitur one hot	Pipeline dari train workshop
Sumber evaluasi	Metadata riset yang disertakan	CV train dan holdout baris 75/25
Tujuan pembelajaran	Mengamati aplikasi yang telah tersedia	Menelusuri model dari data sampai UI

Folder versi 1 mempertahankan source, bobot dan dependensi asal. Versi 2 memperlihatkan hasil yang dapat dibangun dari studi kasus paket. Bobot dan angka evaluasi keduanya tidak dipertukarkan. Pengujian pemuatan model membuktikan program dapat menghasilkan keluaran, bukan verifikasi ulang hasil penelitian sumber.

Menggunakan model melalui Streamlit

Streamlit menyediakan antarmuka untuk aplikasi Python. Dalam contoh ini, pengguna mengisi formulir, aplikasi menyusun DataFrame, pipeline mengubah data menjadi fitur, lalu model menghasilkan kelas dan skor. Alur inferensi mengikuti preprocessing yang sudah disimpan.

Pola program aplikasi

```
import pandas as pd
import streamlit as st
import joblib
model = joblib.load('models/pe_workshop_pipeline.joblib')
# profile berisi seluruh fitur dengan nama yang sesuai.
row = pd.DataFrame([profile])
if st.button('Jalankan model'):
  st.write(model.predict(row)[0])
```

Cuplikan ini menunjukkan pola dasar. app.py dalam paket berisi formulir lengkap delapan fitur, pemuatan model, pemeriksaan versi dan penyajian hasil. Aplikasi tidak melatih encoder baru dari input formulir. Setiap prediksi menggunakan pipeline yang sama dengan eksperimen.

Menjalankan versi hasil studi kasus

Aktifkan environment praktikum, kemudian jalankan perintah berikut dari folder utama paket. Instalasi dependensi dilakukan sebelum kegiatan praktik.

```
python -m pip install -r requirements.txt
python -m streamlit run deployment/versi_2_hasil_studi_kasus/app.py
```

Buka alamat localhost yang ditampilkan terminal. Masukkan nilai sintetis dari baris pertama contoh_input.csv dan bandingkan kelas serta skor dengan notebook 08. Mengubah satu atribut belum tentu mengubah skor, karena kedua masukan dapat mencapai daun pohon yang sama.

Menjalankan versi asli

Versi asli menggunakan environment terpisah sesuai environment.yml dan requirements.txt pada deployment/versi_1_Contoh. Dependensi sumber menetapkan Python 3.10, scikit-learn 1.2.0 dan XGBoost 1.7.5. Bobot model sudah disertakan di models.

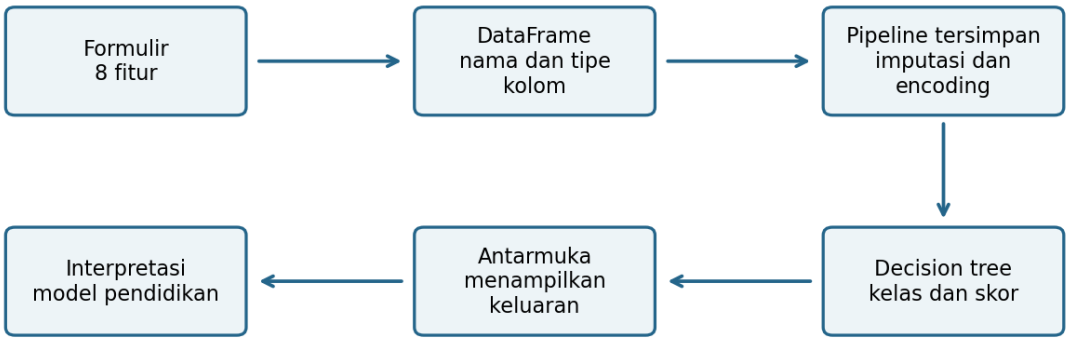
```
conda env create -f environment.yml
conda activate petab2conv
python diagnose_model.py
python -m streamlit run app.py
```

Perintah versi asli dijalankan setelah masuk ke folder versi_1_Contoh. diagnose_model.py memeriksa pemuatan artefak, kontrak fitur dan contoh inferensi. README masing-masing folder menjelaskan langkah lengkap.

Deployment lokal

Pada workshop, aplikasi berjalan lokal. Ini sudah memperlihatkan hubungan data, pelatihan dan penggunaan model. Hosting publik memerlukan pekerjaan lanjutan tentang akses, data, pemantauan dan pemeliharaan. Notebook dapat dibaca mandiri tanpa menjalankan server; output contoh telah disimpan.

Menelusuri satu input dari notebook ke aplikasi



Gambar 12 Alur inferensi versi 2. Pipeline yang sama menghubungkan hasil training dengan masukan formulir.

Langkah 1 Muat artefak dan contoh input

Notebook 08 pada deployment/Notebook_08 memuat pipeline dan contoh_input.csv dari folder deployment versi 2. Contoh input berisi tiga profil sintetis untuk demonstrasi. Setiap baris input menyediakan delapan fitur sesuai model card. Target tidak disertakan karena pada penggunaan model kita sedang meminta prediksi target tersebut.

Langkah 2 Pertahankan kontrak data

USIA dan PARITAS merupakan angka; enam kolom lain mengikuti kategori yang digunakan saat training. Nama kolom, arti atribut dan satuannya harus konsisten. validate_features memeriksa kolom wajib, kolom tambahan, tipe numerik, rentang dan kategori. Input yang lolos validasi dikirim ke pipeline tanpa one hot manual. Formulir meminta seluruh nilai terisi; nilai kosong hanya diizinkan pada demonstrasi imputasi notebook.

Langkah 3 Baca kelas dan skor secara terpisah

```
kelas = model.predict(row)[0]
pos = list(model.classes_).index(1)
skor_pe = model.predict_proba(row)[0, pos]
```

predict menghasilkan kelas pilihan model. predict_proba menyediakan skor untuk setiap kelas dengan urutan classes_. Mengambil kolom kedua tanpa memeriksa kelas dapat menyebabkan salah tafsir pada model lain. Skor pohon berbobot dalam latihan ini belum merupakan risiko medis individual yang terkalibrasi.

Langkah 4 Cocokkan hasil untuk input identik

Isi formulir dengan nilai sintetis dari baris contoh yang sama. Bandingkan kelas dan skor dengan notebook 08. Jika hasil berbeda, periksa isi input, file model, preprocessing, versi library dan cache aplikasi. Mengubah satu fitur belum tentu mengubah prediksi jika sampel tetap jatuh pada daun yang sama.

Versi 1 memiliki sembilan prediktor kategorikal dan kontrak 21 fitur; versi 2 memiliki delapan prediktor dengan pipeline yang diekspor dari studi kasus. Keduanya adalah contoh aplikasi berbeda. Hasil satu versi tidak dipakai untuk memverifikasi angka evaluasi versi lainnya.

Hubungkan ke notebook 07 dan 08. Jelaskan mengapa yang disimpan adalah pipeline lengkap. File hasil pada studi_kasus/model disalin ke models pada deployment versi 2.

Pengayaan klasifikasi teks dari representasi ke model

Notebook 09 memperpanjang gagasan representasi pada notebook 02 menjadi supervised classification. Dataset_Sentimen_Emosi.csv memuat 905 baris sumber. Sebelum belajar, notebook memeriksa teks dan label, menangani baris yang tidak lengkap, serta menghindari pengulangan teks yang sama di train dan test.

Langkah 1 Periksa contoh dan konflik label

Jika teks identik memiliki dua label berbeda, model tidak mendapat jawaban yang konsisten untuk contoh itu. Notebook mengecualikan teks yang berkonflik label, lalu menghapus duplikasi teks identik dengan label sama sebelum split. Jumlah setelah pembersihan dibaca dari output notebook, bukan diasumsikan tetap 905.

Langkah 2 Pelajari kosakata pada train

TfidfVectorizer dalam pipeline mempelajari kosakata dan IDF ketika fit dijalankan. max_features = 2000 membatasi banyaknya istilah. Saat test diproses, kata yang tidak masuk kosakata train tidak membentuk kolom baru. Ini menjaga arti dan urutan fitur tetap konsisten.

Langkah 3 Pilih konfigurasi pada train

Notebook 09 membandingkan model awal, representasi hitungan dan kandidat TF-IDF dengan Naive Bayes pada train. GridSearchCV menguji 12 kombinasi ngram_range, min_df dan alpha dengan macro-F1. Model terpilih baru dinilai pada holdout setelah pencarian selesai.

Multinomial Naive Bayes merupakan classifier ringan yang dapat digunakan pada representasi hitungan atau bobot kata nonnegatif seperti TF-IDF. Intuisinya adalah menggabungkan bukti dari fitur terhadap kelas dengan asumsi penyederhanaan ketergantungan fitur. Asumsi tersebut tidak berarti kata dalam bahasa alami benar-benar saling bebas.

Langkah 4 Baca kesalahan dengan konteks bahasa

Periksa pesan yang salah diprediksi. Sarkasme, negasi, singkatan, bahasa campuran dan konteks yang hilang dapat membuat representasi kata kurang memadai. Menambah stemming atau menghapus stopword tidak otomatis memperbaiki hasil; setiap perubahan perlu dievaluasi pada skema validasi yang sesuai.

Latihan pengayaan

Tulis dua kalimat dengan kata yang hampir sama tetapi makna berbeda, misalnya “layanan ini bagus” dan “layanan ini tidak bagus”. Amati apakah model membedakannya. Hasil pada dua kalimat ini adalah demonstrasi perilaku, bukan ukuran kinerja keseluruhan.

Hubungkan ke notebook 09. Bandingkan pipeline teks dengan pipeline tabular. Keduanya menjaga transformasi dan classifier dalam satu objek, tetapi transformasi fiturnya berbeda.

Latihan pemahaman dan pengembangan

Latihan 1 Menetapkan fitur dan target

Sebuah layanan ingin mengelompokkan pesan menjadi akademik, keuangan dan teknis. Apa unit sampel, fitur dan targetnya? Apa informasi yang perlu tersedia sebelum training?

Pembahasan: unit sampel adalah satu pesan; fitur dapat berupa representasi teks; target adalah kategori layanan. Training memerlukan contoh pesan dengan label yang definisinya konsisten. Data pribadi yang tidak diperlukan untuk tugas tidak perlu menjadi fitur.

Latihan 2 Menemukan kebocoran data

Seorang pengembang menghitung median dari seluruh data, mengisi nilai kosong, kemudian melakukan train-test split. Apa yang perlu diubah?

Pembahasan: lakukan split sebelum mempelajari median. Tempatkan imputer dalam pipeline, fit pada train dan transform pada test. Prinsip yang sama berlaku pada scaling, kosakata TF-IDF dan seleksi fitur yang mempelajari data atau label.

Latihan 3 Membaca confusion matrix

Model menghasilkan TP = 8, FN = 2, FP = 12 dan TN = 78. Hitung accuracy, precision, recall dan specificity.

Pembahasan: jumlah sampel 100; accuracy = 86/100 = 0,86; precision = 8/20 = 0,40; recall = 8/10 = 0,80; specificity = 78/90 ≈ 0,867. Delapan dari sepuluh positif ditemukan, tetapi hanya empat dari sepuluh prediksi positif yang benar.

Latihan 4 Memahami representasi

Sebuah citra RGB berukuran 32 × 32 diratakan menjadi vektor. Berapa panjang vektornya? Apakah bentuk numerik ini setara maknanya dengan tabel 3.072 fitur pelanggan?

Pembahasan: panjangnya 32 × 32 × 3 = 3.072. Dimensi yang sama tidak membuat maknanya sama. Posisi piksel dan kanal warna berbeda dengan atribut pelanggan; model dan preprocessing perlu menyesuaikan struktur tersebut.

Latihan 5 Menelusuri model ke aplikasi

Hasil aplikasi berbeda dari notebook untuk input yang sama. Sebutkan tiga hal yang perlu diperiksa.

Pembahasan: periksa apakah artefak model sama, apakah nama dan tipe fitur sesuai, dan apakah preprocessing memakai parameter hasil train. Periksa juga versi library dan kemungkinan aplikasi masih menyimpan model lama dalam cache setelah artefak diganti.

Proyek kecil setelah workshop

Pilih masalah di kampus atau lingkungan kerja. Tuliskan pengguna, unit sampel, fitur, target, waktu ketersediaan informasi, metrik dan penggunaan hasil. Susun rencana enam tahap CRISP-DM. Mulailah dari baseline yang sederhana, kemudian jelaskan perbaikan apa yang benar-benar dibutuhkan.

Istilah dan sumber belajar

Istilah	Arti ringkas
Dataset	Kumpulan sampel beserta atributnya
Feature space	Ruang representasi yang digunakan model
Imputasi	Pengisian nilai kosong dengan aturan tertentu
Encoding	Pengubahan representasi, misalnya kategori menjadi indikator
Scaling	Penyesuaian skala numerik berdasarkan aturan yang dipelajari
Pipeline	Rangkaian preprocessing dan model dalam satu alur
Baseline	Metode sederhana sebagai pembanding
Overfitting	Model terlalu menyesuaikan data latih sehingga kurang baik pada data baru
Data leakage	Informasi yang seharusnya tidak tersedia ikut memengaruhi pengembangan
Inferensi	Penggunaan model terlatih untuk menghasilkan prediksi
Model card	Dokumentasi model, data, evaluasi dan batas penggunaan

Bacaan pendamping

Bacaan pendamping mencakup Introduction to Machine Learning, Metode Learning, Preprocessing Data, Data Acquisition and Data Understanding, Data Analysis, Data Preparation Training Objectives and Performance Evaluation, Kasus F-measure, Klasifikasi Tabular dengan K-NN, serta TextPreprocessing. Materi tersebut membahas dasar konsep yang diterapkan pada notebook.

Untuk memperdalam pembelajaran, lanjutkan ke validasi lintas kelompok atau waktu, kalibrasi skor, serta pemodelan citra dengan ANN dan CNN. Hubungkan setiap metode dengan masalah, jenis representasi dan desain evaluasi yang sesuai.

Dokumentasi resmi

- IBM. CRISP-DM Help Overview.
  https://www.ibm.com/docs/en/spss-modeler/saas?topic=dm-crisp-help-overview
- scikit-learn. Common pitfalls and recommended practices.
  https://scikit-learn.org/stable/common_pitfalls.html
- scikit-learn. Pipelines and composite estimators.
  https://scikit-learn.org/stable/modules/compose.html
- Streamlit. streamlit run.
  https://docs.streamlit.io/develop/api-reference/cli/run

Dokumentasi tersebut menjadi rujukan teknis untuk kerangka proses, pencegahan kebocoran data, pipeline dan menjalankan aplikasi. Batas interpretasi studi kasus dibahas bersama data dan hasilnya agar kode dapat dibaca dengan konteks yang tepat.